

Módulo 10 – Capítulo 06 – Sección 01

Definición de pipeline de MLOps: desde datos hasta predicción en producción

Un pipeline de MLOps es un flujo de trabajo automatizado y reproducible que encadena todas las etapas necesarias para transformar datos crudos en un modelo de ML desplegado en producción y monitoreado de forma continua. La palabra clave es "reproducible": dado el mismo conjunto de inputs (datos, código, configuración), el pipeline debe poder re-ejecutarse de principio a fin y producir resultados comparables o idénticos. Esta propiedad distingue un pipeline de MLOps de un script de Python que ejecuta las mismas etapas secuencialmente: el pipeline gestiona los fallos de forma granular, registra los inputs y outputs de cada etapa para auditabilidad, y puede re-ejecutarse desde cualquier punto de fallo sin perder el trabajo completado antes de ese punto.

La estructura estándar de un pipeline de MLOps se articula en seis etapas que corresponden a las responsabilidades de los componentes de plataforma descritos en los capítulos anteriores. La **ingesta y validación de datos** conecta con el feature store y el sistema de data quality del Capítulo 04: los datos entran al pipeline desde el feature store offline y son validados por Great Expectations antes de continuar. La **ingeniería de features** materializa las transformaciones adicionales específicas del modelo y registra las URIs de las features en el run de MLflow para trazabilidad. El **entrenamiento del modelo** lanza el job de GPU con tracking automático de hiperparámetros y métricas en MLflow, con checkpointing periódico en S3 para tolerancia a fallos. La **evaluación y validación offline** compara el modelo candidato contra el modelo champion en producción usando eval datasets con ground truth: es el gate de calidad que decide si el modelo avanza o se rechaza. La **publicación al registry** registra el modelo en MLflow con todos sus metadatos de linaje si supera los gates. El **despliegue y monitoreo** actualiza el endpoint de serving y activa el monitoreo de drift descrito en el Capítulo 05.

La diferencia operacional más importante entre un pipeline gestionado por un orquestador y un script de Python es la **gestión de fallos**: cuando el step de training falla en la hora 8 de un job de 10 horas por un spike de memoria transitorio, un script de Python requiere reiniciar

todo el proceso desde cero; un pipeline gestionado por KubeFlow o Prefect puede reintentar solo el step fallido con el checkpoint más reciente como punto de partida, preservando el cómputo de los primeros 8 pasos. Esta tolerancia a fallos no es solo una comodidad: es lo que hace que los pipelines de larga duración sean prácticos en sistemas de producción donde los jobs de varias horas son comunes.

El pipeline de MLOps también habilita la **auditabilidad completa** de cada proceso de entrenamiento. Cada ejecución del pipeline registra en el sistema de tracking (MLflow) los inputs exactos que consumió (dataset con su hash, configuración con su versión, código con su commit SHA), los outputs que produjo (artefactos del modelo, métricas de evaluación), y el estado de cada etapa (completado, fallido, saltado). Esta trazabilidad permite responder retroactivamente a cualquier pregunta sobre cómo se llegó a un modelo específico en producción, sin depender de la memoria del equipo que lo creó.

Componentes técnicos de un pipeline de MLOps

- **Data validation step:** Great Expectations o Deequ validan schema, completitud y distribución; el pipeline falla explícitamente con mensaje accionable si los datos no cumplen las expectativas; fallar rápido es siempre mejor que fallar tarde.
- **Feature engineering step:** cálculo de features con Spark o pandas, materialización al feature store offline, y registro de las URIs de features en el MLflow run para trazabilidad; conecta el pipeline con el componente de feature store.
- **Training step:** job de entrenamiento distribuido (PyTorch DDP, DeepSpeed) con logging automático de métricas, checkpointing periódico en S3, y registro final en MLflow al completar exitosamente.
- **Evaluation gate:** comparación automática de métricas del nuevo modelo vs el modelo champion en producción; el pipeline solo avanza si el nuevo modelo supera el umbral definido (ej. `accuracy_new > accuracy_champion * 0.99`); el gate rechaza, no solo alerta.
- **Deployment step:** actualización del InferenceService en Kubernetes vía kubectl o ArgoCD, con canary routing inicial (5-10% del tráfico) y rollback automático si las métricas de producción se degradan en las primeras horas.

Nota del Arquitecto: La diferencia entre un pipeline de MLOps y un script es la resistencia a fallos y la trazabilidad. Un pipeline bien construido puede fallar en cualquier etapa, ser reparado y re-ejecutarse desde el punto de fallo sin perder el trabajo previo. Esto no es un detalle técnico menor: en pipelines de entrenamiento de

LLMs que duran días, la capacidad de recuperarse de un fallo transitorio sin reiniciar desde cero puede ahorrar decenas de miles de dólares de GPU compute.

El pipeline de MLOps es la infraestructura que convierte el proceso de mejora de modelos de un esfuerzo manual y artesanal en un proceso sistemático y automatizable. La siguiente sección examina los orquestadores —Airflow, Prefect, Kubeflow Pipelines— que implementan estos pipelines y los factores que determinan cuál es la elección correcta para cada contexto.

Módulo 10 – Capítulo 06 – Sección 02

Orquestadores de pipelines: Airflow, Prefect y Kubeflow Pipelines

Los orquestadores de pipelines de MLOps comparten una funcionalidad central —gestionar la ejecución de flujos de trabajo como grafos acíclicos dirigidos (DAGs) con dependencias entre tareas, reintentos automáticos, y registro de ejecuciones— pero difieren significativamente en su modelo de programación, sus primitivas de ejecución y su integración con el ecosistema de ML. La elección del orquestador correcto no es solo una decisión técnica: es una decisión sobre la cultura de desarrollo de pipelines que el equipo quiere adoptar. Un orquestador que prioriza la potencia del operador pero tiene una curva de aprendizaje alta puede resultar en que los ML Engineers delegan toda la gestión de pipelines al equipo de plataforma en lugar de operar sus propios pipelines. Un orquestador con mejor developer experience pero menos flexibilidad puede generar workarounds cuando se enfrentan casos de uso avanzados.

Apache Airflow es el orquestador más adoptado en la industria con el ecosistema de operadores más amplio: `KubernetesPodOperator` para ejecutar cualquier tarea en un Pod de Kubernetes, `SparkSubmitOperator` para jobs de Spark, `DatabricksRunNowOperator` para notebooks de Databricks, y decenas más para servicios de AWS, GCP y Databricks. Esta amplitud de integración es su principal ventaja: organizaciones con pipelines de datos heterogéneos (ML + dbt + ETL + análisis) pueden centralizar la orquestación en Airflow sin cambiar de herramienta. Sus debilidades son igualmente bien documentadas: el modelo de programación de DAGs en Python tiene una curva de aprendizaje pronunciada porque el código del DAG se evalúa en el scheduler sin ejecutarse (lo que confunde a los desarrolladores acostumbrados al flujo normal de Python), el testing local es complejo porque requiere un entorno similar al de producción, y los cambios en el código de los DAGs en producción pueden causar comportamientos inesperados si no se gestionan cuidadosamente.

Prefect adopta un modelo de programación más cercano al código Python estándar, usando decoradores que transforman funciones normales en tareas y flujos del pipeline: `@task` convierte una función en una tarea con retry logic y logging automático, y `@flow` convierte una función en un flujo orquestable. La diferencia operacional respecto a Airflow es

significativa: el testing local de un Prefect flow es trivial porque `if __name__ == "__main__":` `my_flow()` ejecuta el flujo exactamente como lo haría el orquestador, sin necesidad de emular el comportamiento del scheduler. El modelo de ejecución distribuida de Prefect separa el servidor central de orquestación (Prefect Cloud o self-hosted) de los workers que ejecutan el cómputo donde está disponible (Kubernetes, ECS, Lambda, o simplemente el entorno local del desarrollador). Prefect es la opción que maximiza la experiencia del ML Engineer como operador de sus propios pipelines.

Kubeflow Pipelines está diseñado específicamente para ML y se integra de forma nativa con el ecosistema de Kubernetes. Los componentes de un pipeline de Kubeflow son contenedores Docker —cualquier proceso que corre en un container puede ser un step del pipeline— y el pipeline se define en Python con el SDK de KFP (Kubeflow Pipelines), generando automáticamente un YAML de Argo Workflows que Kubernetes ejecuta. La integración nativa con el ecosistema Kubeflow es su diferenciador: Katib para hyperparameter tuning automático, Training Operator para distributed training con PyTorchJob o TFJob, y el Metadata Store (MLMD) para tracking de artefactos son componentes que se integran de forma nativa con los pipelines de KFP. La desventaja es que Kubeflow Pipelines requiere un cluster de Kubernetes dedicado o compartido, y la operación del ecosistema completo de Kubeflow tiene una complejidad operativa significativa.

Comparación técnica de orquestadores

- **Apache Airflow:** ecosistema más amplio, operadores para decenas de servicios, scheduling basado en cron con backfill nativo; complejidad operativa alta, curva de aprendizaje pronunciada; mejor para organizaciones con múltiples tipos de pipelines de datos, no solo ML.
- **Prefect:** modelo de programación Python puro con decoradores, testing local trivial, UI moderna, deployment flexible (Kubernetes, Docker, Process); menor overhead operativo que Airflow; mejor para equipos que priorizan la experiencia del ML Engineer como operador de sus propios pipelines.
- **Kubeflow Pipelines:** integración nativa con Kubernetes y ecosistema Kubeflow (Katib, Training Operator), componentes reutilizables como contenedores, MLMD para tracking de artefactos; requiere cluster Kubernetes dedicado; mejor para plataformas Kubernetes-first con alto volumen de training jobs distribuidos.
- **Criterio de selección:** Airflow para organizaciones con múltiples tipos de pipelines de datos que necesitan centralización; Prefect para equipos que priorizan developer

experience y self-service de ML Engineers; Kubeflow para plataformas Kubernetes-nativas con distributed training como caso de uso central.

- **Testing de pipelines:** independientemente del orquestador elegido, cada step del pipeline debe ser testeable en aislamiento con PyTest; los tests de integración deben ejecutarse contra un entorno de staging con datos sintéticos, no sobre datos de producción.

Nota del Arquitecto: La elección del orquestador determina la cultura de desarrollo de pipelines: priorizar la experiencia del ML Engineer (Prefect) sobre la potencia del operador de datos (Airflow) resulta en mayor adopción y menos pipelines abandonados en favor de scripts ad-hoc. Si los ML Engineers no pueden operar sus propios pipelines sin ayuda constante del equipo de plataforma, el orquestador no está cumpliendo su función habilitadora.

La elección del orquestador es una decisión de largo plazo: migrar de un orquestador a otro después de tener decenas de pipelines en producción es costoso en tiempo de ingeniería y en riesgo operacional. La siguiente sección examina el continuous training —el proceso que cierra el ciclo de MLOps al reentrenar automáticamente los modelos cuando su calidad en producción degrada— y los riesgos específicos de feedback loops autoinducidos que hacen del CT una práctica que requiere diseño cuidadoso.

Módulo 10 – Capítulo 06 – Sección 03

Continuous training: reentrenar automáticamente cuando la calidad cae por debajo del umbral

El continuous training (CT) es la práctica de reentrenar automáticamente un modelo cuando se detecta que su calidad en producción ha caído por debajo de un umbral predefinido, completando el ciclo de MLOps desde el monitoreo hasta la actualización del modelo en producción sin intervención manual. CT es la diferencia entre un sistema de IA que mejora continuamente con el tiempo y uno que se degrada gradualmente hasta que alguien recuerda que necesita ser actualizado. Sin embargo, CT es también la práctica que introduce los riesgos más sutiles y peligrosos del MLOps: el **distributional shift autoinducido**, donde el modelo en producción contamina los datos de entrenamiento del modelo siguiente creando espirales de degradación que son difíciles de detectar y más difíciles de corregir.

La implementación del CT requiere tres componentes integrados. El **sistema de monitoreo** del Capítulo 05 emite una señal cuando la calidad degrada: un drift score que supera el umbral $PSI > 0.2$ en features críticas, o una accuracy rolling de 7 días que cae por debajo del umbral definido, o la acumulación de suficientes nuevas muestras etiquetadas para justificar un reentrenamiento. El **sistema de triggers** convierte esa señal en el inicio de un pipeline de reentrenamiento: un webhook de Evidently AI puede llamar directamente a la API del orquestador para lanzar el pipeline, o un job periódico puede evaluar las condiciones de reentrenamiento y dispararlo cuando se cumplan. El **pipeline de reentrenamiento completamente automatizado** ejecuta desde la ingesta de datos hasta el despliegue sin intervención humana si el nuevo modelo pasa los gates de calidad automáticos.

El riesgo del distributional shift autoinducido

El riesgo más sutil y más peligroso del CT no está en la automatización del reentrenamiento en sí, sino en el origen de los datos de reentrenamiento. Cuando un modelo M_n está en producción, sus predicciones influyen directamente en el comportamiento que los usuarios exhiben, en las decisiones que los sistemas toman, y por tanto en los datos que el sistema recopila. Los datos de producción generados bajo la influencia de M_n no son una muestra

aleatoria e independiente de la distribución real del problema: son datos sesgados por las predicciones del modelo que los generó.

Un ejemplo concreto: un modelo de recomendación de contenido M_n decide qué artículos muestra a cada usuario. Los artículos que M_n recomienda son los que los usuarios ven, hacen clic, y con los que interactúan. El dataset de reentrenamiento M_{n+1} se construye sobre estas interacciones —pero las interacciones ocurrieron solo con los artículos que M_n eligió mostrar, no con todos los artículos disponibles. M_{n+1} aprende que los artículos que M_n recomendaba son los "buenos" (porque recibieron clicks), reforzando el sesgo de selección del modelo anterior. Cada ciclo de reentrenamiento amplifica el sesgo: M_{n+1} restringe más las recomendaciones, M_{n+2} aprende sobre esa distribución aún más estrecha, y el sistema converge en una espiral de degradación que desde el punto de vista de las métricas internas (click-through rate) puede parecer estable o incluso mejorar, mientras que desde el punto de vista de la diversidad y la calidad real del sistema está empeorando.

El distributional shift autoinducido es especialmente peligroso porque las métricas de calidad convencionales no lo detectan: el modelo M_{n+1} puede tener un accuracy superior en el test set (que también fue generado bajo la influencia de los modelos anteriores) mientras que el comportamiento del sistema se degrada desde perspectivas externas como diversidad de recomendaciones, equidad entre subgrupos, o cobertura del espacio de items disponibles.

Estrategias de mitigación

La primera estrategia es mantener un **holdout de distribución original**: un conjunto de datos de referencia con la distribución inicial del problema (datos pre-deployment o datos recopilados con política de exploración aleatoria) que se incluye en cada ciclo de reentrenamiento como ancla de diversidad. Este holdout previene que el modelo olvide completamente las regiones del espacio de features que ya no recibe feedback de producción.

La segunda estrategia es el **mixing ratio controlado**: en lugar de reentrenar exclusivamente sobre datos de producción recientes, mezclar los datos históricos (del dataset original de entrenamiento) con los datos de producción en una proporción configurable (ej. 70% histórico, 30% reciente). El mixing ratio debe ajustarse según la velocidad del concept drift: si el drift es rápido, se necesita más peso en datos recientes; si el drift es lento, más peso en datos históricos preserva la estabilidad del modelo.

La tercera estrategia es el **monitoreo de la distribución del training set** entre ciclos de reentrenamiento: comparar la distribución de las features del training set de M_{n+1} con la del training set de M_n usando los mismos tests estadísticos que se usan para detectar drift

en producción. Si la distribución del training set está derivando de un ciclo al siguiente, es una señal de que el loop de retroalimentación está activo y las estrategias de mitigación son insuficientes.

Aspectos técnicos del continuous training

- **Triggers de reentrenamiento:** basados en tiempo (cada N días), en datos (cuando se acumulan N nuevas muestras etiquetadas), o en calidad (cuando el drift score o las métricas de producción superan el umbral); los triggers en calidad son los más efectivos pero requieren el sistema de monitoreo del Capítulo 05.
- **Data collection para reentrenamiento:** pipeline que combina el dataset histórico con nuevas muestras de producción en mixing ratio configurable; el holdout de distribución original se incluye siempre; la distribución del training set se registra en MLflow para comparación entre ciclos.
- **Gates de aprobación automáticos:** el pipeline de CT despliega el nuevo modelo solo si `accuracy_new > accuracy_champion * 0.99` Y las métricas de producción en canary (primeras N horas) no muestran degradación significativa; el gate doble previene regresiones de calidad offline y de producción.
- **Rollback automático:** si las métricas de producción se degradan dentro de las primeras 1-6 horas después del despliegue automático, el sistema revierte al modelo anterior sin intervención humana; el rollback debe ser probado periódicamente, no solo diseñado.
- **Monitoreo de loops de reentrenamiento:** alertas cuando el pipeline de CT se ejecuta con frecuencia inusualmente alta (posible señal de concept drift severo o distributional shift autoinducido descontrolado) o cuando la distribución del training set difiere significativamente del ciclo anterior.

Nota del Arquitecto: El continuous training sin validación robusta del dataset de reentrenamiento y sin estrategias de mitigación del distributional shift autoinducido es más peligroso que no tener CT. El loop de retroalimentación entre el modelo en producción y los datos de entrenamiento del siguiente modelo es una propiedad estructural de cualquier sistema que aprende de sus propias predicciones: no desaparece ignorándolo, solo se vuelve invisible hasta que el daño es difícil de revertir.

El continuous training efectivo requiere tanto la automatización técnica del ciclo de reentrenamiento como el diseño cuidadoso de los datos con los que se reentrena. La siguiente

sección examina el pipeline testing: cómo validar cada etapa del pipeline antes de llevarla a producción, garantizando que el sistema de CT en sí es confiable y no produce comportamientos inesperados.

Módulo 10 – Capítulo 06 – Sección 04

Pipeline testing: validar cada etapa antes de promover a producción

Un pipeline de MLOps sin tests es una caja negra que produce modelos de forma no reproducible: puede funcionar la primera vez que se ejecuta, la décima vez, y fallar en la undécima de formas que nadie puede diagnosticar rápidamente. El testing de pipelines de MLOps adopta las prácticas de testing de software —unit, integration, end-to-end— adaptadas a las características específicas de los sistemas de ML donde los datos no son fixtures estáticos sino distribuciones variables, los modelos tienen comportamiento probabilístico no determinista, y las métricas de calidad tienen umbrales relativos en lugar de valores exactos.

Los **unit tests** en un pipeline de MLOps validan cada función de transformación de datos y feature engineering de forma aislada con inputs sintéticos. La lógica es la misma que en software convencional: si la función `transform_user_age(birthdate)` tiene un bug que produce edades negativas para fechas futuras, ese bug debe detectarse en el unit test antes de llegar al pipeline de entrenamiento. Ejemplos específicos: `assert transform_user_age(birthdate=date(1990,1,1)) == 35`, `assert feature_encoding(category="unknown") == 0` (el valor esperado para categorías fuera del vocabulario de entrenamiento), `assert normalize_text(" hello WORLD ") == "hello world"`. Los unit tests de pipeline deben ejecutarse en el mismo entorno de CI que los unit tests del código de la aplicación: si requieren GPU, datos de producción, o servicios externos para ejecutarse, no son unit tests efectivos.

Los **integration tests** validan que las etapas del pipeline se conectan correctamente entre sí. Los puntos de integración más críticos en un pipeline de MLOps son: la compatibilidad del schema de output de la etapa de features con el schema de input esperado por el training step (una incompatibilidad aquí solo se detecta en la ejecución real del pipeline si no se testa antes), la compatibilidad del modelo serializado con el formato que el serving layer espera cargar (un modelo guardado en el formato incorrecto o con dependencias de versión incorrectas solo falla cuando se intenta desplegar), y la validez del schema de respuesta del endpoint desplegado contra el schema que la aplicación cliente espera consumir.

Los **model validation tests** son específicos de ML y no tienen equivalente en software convencional. El test más importante es la comparación de métricas offline del modelo candidato contra el modelo champion: `assert accuracy_new >= accuracy_champion * 0.99` con un margen de tolerancia explícito que permita variación estadística sin rechazar mejoras reales. Los **tests de invarianza** verifican que la predicción no cambia significativamente si se perturban features que deberían ser irrelevantes: el modelo de pricing de un producto no debería cambiar su predicción si el `user_id` cambia pero todas las features de comportamiento son idénticas. Los **tests de fairness** verifican que la métrica de calidad no difiere más del umbral permitido entre subgrupos protegidos: si el modelo de scoring tiene accuracy de 92% para usuarios masculinos y 78% para usuarios femeninas, el test de fairness detecta esa disparidad antes de que llegue a producción.

Los **load tests pre-producción** son el último gate antes del despliegue a producción. Locust o k6 ejecutan tráfico sintético contra el endpoint en staging para verificar que el modelo cumple los SLOs de latencia (TTFT, TPOT para LLMs) bajo la carga máxima esperada. El error más común es hacer load tests con carga mucho menor que el pico de producción previsto: si el endpoint en staging se testea con 50 RPS y el pico de producción es 500 RPS, el load test no ofrece ninguna garantía real sobre el comportamiento en producción.

Tipos de tests en pipelines de MLOps

- **Data validation tests:** Great Expectations ejecutadas automáticamente en el step de ingesta; el pipeline falla con mensaje de error específico si el schema cambia inesperadamente o si la tasa de nulos supera el umbral; los failures deben ser informativas, no genéricas.
- **Feature engineering unit tests:** PyTest que validan cada función de transformación con inputs edge case (valores nulos, strings vacíos, valores fuera de rango, categorías OOV); aislados de infraestructura y ejecutables en CI sin GPU ni servicios externos.
- **Model evaluation tests:** comparación automática de métricas del modelo candidato vs champion; el gate rechaza el modelo si `accuracy_new < accuracy_champion * 0.99` o si cualquier métrica de fairness cae por debajo del umbral definido; gate de calidad, no solo alerta.
- **Serving compatibility tests:** verificar que el modelo serializado puede ser cargado por el serving layer (KServe, Triton) sin errores, y que el endpoint retorna respuestas con el schema correcto para inputs de ejemplo representativos del tráfico real.
- **Load tests pre-producción:** Locust o k6 sobre el staging endpoint verificando que cumple los SLOs de latencia bajo la carga máxima esperada antes de promover a

producción; la carga de test debe ser al menos el pico de producción previsto.

Nota del Arquitecto: *En un pipeline de MLOps, un test que no puede ejecutarse en el entorno de CI —por depender de datos de producción, GPU cara o servicios externos— no es un test efectivo: siempre se omitirá bajo presión de tiempo. Los tests críticos deben poder ejecutarse en un entorno reproducible y aislado, con datos sintéticos que capturen los casos edge más importantes. La regla práctica: si el test tarda más de 5 minutos en ejecutarse, no se ejecutará en cada PR.*

El testing de pipelines de MLOps es la práctica que convierte la confianza intuitiva en el sistema en confianza técnica verificable. Un pipeline bien testado es uno que el equipo puede desplegar el viernes a las 5pm con la misma serenidad con la que lo desplegaría el martes por la mañana. La siguiente sección examina cómo GitOps provee el mecanismo de control de versiones y despliegue declarativo que hace que esa confianza sea sistémica en lugar de dependiente de la experiencia del ingeniero que ejecuta el despliegue.

Módulo 10 – Capítulo 06 – Sección 05

GitOps para MLOps: infraestructura como código y despliegue declarativo

GitOps aplica el principio de que el estado deseado de toda la infraestructura y configuración del sistema se describe declarativamente en un repositorio Git, y un operador automático (ArgoCD, Flux) se encarga de reconciliar continuamente el estado real del cluster con el estado declarado en Git. Este principio, que en el contexto de aplicaciones web convencionales ya es una práctica estándar, adquiere una dimensión adicional en el contexto de MLOps: la versión del modelo que está en producción, los parámetros de scaling del serving layer, y la configuración del pipeline de monitoreo son parte del estado del sistema que debe estar bajo control de versiones con la misma rigurosidad que el código de la aplicación.

En el contexto de MLOps, GitOps cubre tres dimensiones complementarias. La primera es la **infraestructura de la plataforma**: la definición de namespaces de Kubernetes, ResourceQuotas, NetworkPolicies e IngressRoutes se describe en Helm charts o manifiestos de Kustomize que viven en Git. Cuando el equipo de plataforma necesita añadir un nuevo namespace para un equipo de AI Engineering, o aumentar la ResourceQuota de GPU de un equipo, el cambio va por un Pull Request con revisión — no por un `kubectl apply` manual que nadie podrá rastrear después.

La segunda dimensión es la **configuración de los modelos en producción**: los InferenceService de KServe con la versión del modelo, los parámetros de scaling (min/max replicas, GPU resources) y los recursos de CPU/GPU se definen en archivos YAML en el repositorio de configuración (config repo), no se modifican directamente con `kubectl`. Un Helm chart parametrizado acepta `model_uri`, `replicas`, `cpu_limit`, `memory_limit`, `gpu_count` como valores configurables: el CI/CD pipeline de entrenamiento actualiza el `values.yaml` con la nueva versión del modelo cuando ésta pasa los gates de calidad, y ArgoCD aplica el cambio automáticamente. El resultado es que el historial de Git contiene exactamente cuándo, quién y qué versión del modelo fue desplegada en producción, incluyendo el PR de code review que aprobó el cambio.

La tercera dimensión es los **pipelines de MLOps**: las definiciones de los DAGs de Airflow o los Prefect Deployments se versionan en Git y se despliegan automáticamente al mergearse al branch principal. Si un PR introduce un bug en la lógica del pipeline que no fue capturado por los tests, `git revert` es el mecanismo de rollback: el orquestador carga automáticamente la versión anterior del DAG. La separación entre el repositorio de código de entrenamiento y el repositorio de configuración de despliegue (config repo) es una práctica importante: el pipeline de CI actualiza el config repo cuando un nuevo modelo pasa los gates de calidad, creando un Pull Request automático que puede ser revisado y aprobado por un humano antes de que ArgoCD aplique el cambio a producción.

El beneficio de GitOps para MLOps se materializa en el momento de un incidente. Cuando un modelo de producción empieza a comportarse de forma inesperada después de un despliegue, la primera pregunta es "¿qué cambió?". Con GitOps, esa pregunta tiene una respuesta en segundos: el historial del config repo muestra exactamente qué versión del modelo fue desplegada, cuándo, y qué PR la autorizó, incluyendo el comentario del revisor que aprobó el cambio. Sin GitOps, esa misma pregunta requiere investigación en logs de kubectl, conversaciones con el equipo y reconstrucción manual de la secuencia de eventos.

Aspectos técnicos de GitOps para MLOps

- **Repository structure:** separar el repo de código de entrenamiento del repo de configuración de despliegue (config-as-code); el pipeline de CI actualiza automáticamente el config repo cuando un nuevo modelo pasa los gates de calidad; la separación establece un punto de control auditable.
- **ArgoCD Application:** recurso de Kubernetes que mapea un directorio del repositorio Git a un namespace del cluster, con sync policy (manual para producción, automático para staging) y configuración de pruning de recursos eliminados.
- **Helm charts para modelos:** un chart parametrizado acepta `model_uri`, `replicas`, `cpu_limit`, `memory_limit` como values; el CI actualiza el `values.yaml` con la nueva versión del modelo; ArgoCD detecta el cambio y aplica el sync.
- **Promotion workflow:** el pipeline de entrenamiento crea un PR automático al config repo con la nueva versión del modelo; un humano aprueba el PR (o el proceso es totalmente automatizado para deployments de bajo riesgo); ArgoCD aplica el cambio creando un audit trail completo en Git.
- **Secret management:** los secretos (API keys, credenciales) no se almacenan en Git sino en HashiCorp Vault o AWS Secrets Manager; External Secrets Operator sincroniza los secretos desde Vault al cluster; las referencias a secretos (no los valores) sí viven en Git.

Nota del Arquitecto: *El repositorio de configuración de GitOps para MLOps debe ser el único origen de verdad sobre qué versión de qué modelo está en producción. Cualquier cambio manual con `kubectl` que no pase por Git debe considerarse una violación de proceso y un riesgo de auditoría: no porque el cambio sea necesariamente incorrecto, sino porque es invisible para el sistema de trazabilidad y no puede ser reproducido ni revertido de forma controlada.*

GitOps provee la trazabilidad, la reproducibilidad y el control de cambios que hacen del despliegue de modelos un proceso confiable y auditable. La sección de cierre de este capítulo sintetiza el argumento central del Capítulo 06: que un pipeline de MLOps bien diseñado no es un lujo sino la precondition para que la mejora continua de modelos sea sostenible a largo plazo.

Módulo 10 – Capítulo 06 – Sección 06

Cierre: un pipeline de MLOps bien diseñado hace que la mejora continua sea automática

El objetivo final de un pipeline de MLOps no es la automatización per se, sino la reducción del tiempo entre "tenemos una nueva versión del modelo que supera al actual" y "esa versión está en producción sirviendo usuarios". Cuando ese ciclo pasa de semanas a horas, y de horas a minutos para los casos de menor riesgo, la organización gana la capacidad de iterar sobre sus modelos con una velocidad que no es posible con procesos manuales: responder a cambios en los datos de producción en horas en lugar de semanas, probar múltiples variantes de modelo simultáneamente con canary deployment, y mantener la calidad de los sistemas de IA de forma sostenible sin que cada actualización requiera un proyecto de infraestructura.

Un pipeline bien diseñado también cambia la dinámica del equipo en una dirección menos obvia pero igualmente importante. Cuando el despliegue es un proceso confiable, automatizado y reversible, los ingenieros dejan de acumular cambios por miedo a que el despliegue falle. En sistemas sin pipelines sólidos, es común que los equipos acumulen semanas de cambios en un branch antes de hacer merge, porque cada deploy es una operación de alto riesgo que puede requerir horas de debugging si algo sale mal. Con un pipeline de MLOps confiable —con tests robustos, gates automáticos y rollback probado— los equipos hacen releases pequeños y frecuentes (continuous delivery): cada cambio pequeño tiene menor riesgo, es más fácil de diagnosticar si algo falla, y el ciclo de feedback es más rápido.

Los pipelines de MLOps, como toda infraestructura de software, requieren mantenimiento activo para seguir siendo efectivos. Las dependencias de Python se actualizan y rompen el build con cambios de API; los schemas de datos de los sistemas fuente cambian sin previo aviso; los umbrales de los gates de calidad necesitan recalibración a medida que los modelos mejoran y los niveles de baseline suben. Un pipeline que no se mantiene acumula deuda técnica que lo convierte gradualmente en un obstáculo: primero los tests se vuelven flakey y el equipo empieza a ignorar sus fallos, luego los gates de calidad se vuelven demasiado laxos porque nadie los ajustó después del último reentrenamiento, y finalmente el pipeline se

convierte en el cuello de botella que el equipo evita en lugar de la infraestructura que lo habilita.

La medida más reveladora de la madurez de un pipeline de MLOps no es su sofisticación técnica sino la confianza del equipo en él: un equipo que usa el pipeline para desplegar el viernes a las 5pm —el escenario que en organizaciones con procesos frágiles es tabú— demuestra que el sistema tiene los tests, los gates y el rollback suficientemente robustos como para que el riesgo del despliegue sea aceptable en cualquier momento, no solo cuando hay suficiente tiempo para revertirlo manualmente si algo sale mal.

Principio rector

Un pipeline de MLOps bien diseñado es aquel que el equipo confía lo suficiente como para hacer un despliegue a producción el viernes a las 5pm: ese nivel de confianza se construye con tests robustos, gates automáticos y un rollback probado que funciona. La confianza no es retórica —es técnica: es el resultado de haber visto el rollback funcionar en staging, de haber visto los gates rechazar un modelo degradado, de haber visto los tests detectar un bug antes de que llegue a producción.

"Make it work, make it right, make it fast."

— Kent Beck, creador de Extreme Programming y Test-Driven Development, describiendo la secuencia correcta de prioridades en ingeniería de software — aplicable directamente a la construcción iterativa de pipelines de MLOps: primero que funcione, luego que funcione bien, luego que funcione rápido.